

Полный конспект к зачёту по C++

Содержание

1 Билет 1. Основы. Базовые типы данных. Указатели, массивы	5
1.1 Базовые типы данных	5
1.1.1 Целочисленные типы	5
1.1.2 Типы с плавающей точкой	5
1.1.3 Символьные типы	5
1.1.4 Логический тип	5
1.1.5 void	5
1.2 Указатели	6
1.2.1 Адресная арифметика	6
1.3 Массивы	6
1.4 Ссылки (кратко)	6
2 Билет 2. Компиляция. Этапы. Препроцессор. ODR. Mangling	7
2.1 Этапы компиляции	7
2.2 Директивы препроцессора	7
2.2.1 #include	7
2.2.2 #define и макросы	7
2.2.3 Include Guard	8
2.3 Объявление vs Определение	8
2.4 One Definition Rule (ODR)	8
2.5 Связывание (Linkage)	8
2.6 inline	9
2.7 Name Mangling (Декорирование имён)	9
3 Билет 3. Классы. RAII. Модификаторы. Специальные методы	9
3.1 Структура класса	9
3.2 Конструкторы и деструкторы	10
3.2.1 Конструкторы	10
3.2.2 Списки инициализации (Member Initializer List)	10
3.2.3 Деструктор	10
3.3 RAII (Resource Acquisition Is Initialization)	10
3.4 Модификаторы доступа	10
3.5 Ключевые слова для членов класса	11
3.5.1 static	11
3.5.2 explicit	11
3.5.3 const для методов	11
3.6 Перегрузка операторов	11
3.7 Специальные методы (Rule of 5)	11
3.8 Идиома Copy-and-Swap	12
4 Билет 4. Работа с памятью. Стек и куча	12
4.1 Модель памяти программы	12
4.2 Стек (Stack)	13
4.3 Куча (Heap / Free Store)	13
4.4 new / delete	13
4.5 operator new / operator delete	13
4.6 malloc / free (из C)	13
4.7 Placement new	14
4.8 std::construct_at / std::destroy_at (C++20)	14

4.9	Проблемы динамической памяти	14
5	Билет 5. Исключения	14
5.1	Механизм исключений	14
5.2	Иерархия стандартных исключений	15
5.3	поexcept	15
5.4	Гарантии безопасности исключений	15
5.5	Альтернативы исключениям	15
6	Билет 6. Шаблоны (Templates)	16
6.1	Зачем нужны шаблоны?	16
6.2	Шаблоны функций и классов	16
6.3	Двухфазный поиск имён (Two-Phase Lookup)	16
6.4	Специализация	17
6.4.1	Полная специализация	17
6.4.2	Частичная специализация (только для классов)	17
6.5	Специализации функций и перегрузки	17
6.6	Variadic Templates (шаблоны с переменным числом аргументов)	17
6.7	Fold Expressions (C++17)	18
7	Билет 7. Трейты и SFINAE	18
7.1	Type Traits (трейты типов)	18
7.2	std::declval	19
7.3	SFINAE (Substitution Failure Is Not An Error)	19
7.3.1	Техники применения SFINAE	20
7.4	if constexpr (C++17)	20
7.5	Tag Dispatching	20
7.6	SFINAE-friendliness	21
8	Билет 8. Контейнеры и итераторы	21
8.1	Контейнеры STL	21
8.1.1	Последовательные контейнеры	21
8.1.2	Ассоциативные контейнеры (на деревьях)	21
8.1.3	Неупорядоченные контейнеры (хеш-таблицы)	21
8.1.4	Контейнеры-адаптеры	21
8.2	Итераторы	22
8.2.1	Категории итераторов (иерархия)	22
8.3	Range-based for	22
8.4	std::iterator_traits	23
8.5	Инвалидация итераторов	23
9	Билет 9. Undefined Behavior (UB)	23
9.1	Что такое UB?	23
9.2	Примеры UB	23
9.3	Strict Aliasing Rule	24
9.4	std::unreachable() (C++23)	24
9.5	assert	24
9.6	Санитайзеры	25
9.7	Отладочная версия стандартной библиотеки	25

10 Билет 10. Вычисления на этапе компиляции	25
10.1 constexpr	25
10.2 consteval (C++20)	25
10.3 constexpr (C++20)	26
10.4 std::is_constant_evaluated() (C++20)	26
11 Билет 11. Перечислимый тип и тип-объединение	26
11.1 enum и enum class	26
11.1.1 enum (C-style)	26
11.1.2 enum class (C++11, scoped enumeration)	26
11.2 union	26
11.3 std::variant (C++17)	27
11.4 std::visit	27
11.5 std::monostate	28
11.6 valueless_by_exception	28
12 Билет 12. Rvalue ссылки и семантика перемещения	28
12.1 Категории значений	28
12.2 Rvalue ссылки	29
12.3 Семантика перемещения	29
12.4 std::move	29
12.5 Избегание копирования: RVO и NRVO	29
12.6 Классы-агрегаты	30
12.7 Ref-спецификация методов	30
13 Билет 13. Perfect Forwarding	30
13.1 Проблема	30
13.2 Universal Reference (Forwarding Reference)	31
13.3 Reference Collapsing (сворачивание ссылок)	31
13.4 Правило вывода типов для forwarding reference	31
13.5 std::forward	31
13.6 Perfect Backwarding	32
13.7 decltype и trailing return type	32
13.8 auto и decltype(auto)	32
14 Билет 14. Наследование и Полиморфизм	32
14.1 Наследование	32
14.2 Slicing (срезка)	33
14.3 Статический и динамический тип	33
14.4 Виртуальные функции	33
14.5 Чисто виртуальные функции и абстрактные классы	34
14.6 Виртуальный деструктор	34
14.7 dynamic_cast и RTTI	34
14.8 Empty Base Optimization (EBO)	34
14.9 [[no_unique_address]] (C++20)	35
14.10 Множественное наследование	35
14.11 Виртуальное наследование	35
15 Билет 15. Пространства имён	35
15.1 namespace	35
15.2 Namespace alias	36
15.3 using-объявление vs using-директива	36
15.4 ADL (Argument Dependent Lookup / Koenig Lookup)	36
15.5 Безымянные (анонимные) пространства имён	36

16 Билет 16. Статический и динамический полиморфизм. Лямбды	37
16.1 Статический полиморфизм	37
16.2 Динамический полиморфизм	37
16.3 Сравнение	37
16.4 Лямбда-выражения	37
16.4.1 Списки захвата	37
16.4.2 Как лямбда устроена внутри?	38
16.4.3 Свойства типа лямбды	38
16.4.4 Generic лямбды (C++14)	38
16.4.5 Шаблонные лямбды (C++20)	38
16.4.6 Лямбды в невычисляемых контекстах (C++20)	38
17 Билет 17. Библиотеки и ABI	38
17.1 Виды библиотек	38
17.1.1 Header-only	38
17.1.2 Статические библиотеки (.a / .lib)	39
17.1.3 Динамические библиотеки (.so / .dll / .dylib)	39
17.2 ABI (Application Binary Interface)	39
17.3 extern ‘C’	39
17.4 PImpl (Pointer to Implementation)	40
17.5 Наследование при PImpl	40

1 Билет 1. Основы. Базовые типы данных. Указатели, массивы

1.1 Базовые типы данных

В C++ типы делятся на несколько категорий:

1.1.1 Целочисленные типы

`short`, `int`, `long`, `long long`. Каждый может быть `signed` (по умолчанию) или `unsigned`.

Стандарт **не фиксирует** конкретные размеры, а задаёт лишь минимальные гарантии:

- `short` — не менее 16 бит
- `int` — не менее 16 бит (на практике почти всегда 32)
- `long` — не менее 32 бит
- `long long` — не менее 64 бит

И гарантирован порядок: $1 = \text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$.

Почему размеры не фиксированы? Потому что C++ проектировался для работы на разных архитектурах (от 8-битных микроконтроллеров до 64-битных серверов). Фиксация размеров привела бы к неэффективности на некоторых платформах. Если нужен точный размер, используют типы из `<stdint.h>`: `int8_t`, `int16_t`, `int32_t`, `int64_t` и их `uint`-аналоги.

Знаковое переполнение — это UB. **Беззнаковое переполнение** — определено стандартом (арифметика по модулю 2^n).

1.1.2 Типы с плавающей точкой

`float` (обычно 32 бита, IEEE 754 single), `double` (обычно 64 бита, IEEE 754 double), `long double` (платформозависим).

Как работает представление? Число хранится в виде $(-1)^s \times m \times 2^e$, где s — знак, m — мантисса, e — экспонента. Из-за конечности мантиссы возникают ошибки округления: $0.1 + 0.2 \neq 0.3$.

1.1.3 Символьные типы

- `char` — 1 байт. Может быть `signed` или `unsigned` (implementation-defined!).
- `wchar_t` — «широкий» символ (размер зависит от платформы: 2 байта на Windows, 4 на Linux).
- `char16_t`, `char32_t` (C++11) — фиксированные размеры для Unicode (UTF-16, UTF-32).
- `char8_t` (C++20) — для UTF-8.

1.1.4 Логический тип

`bool`: хранит `true` или `false`. Гарантированно может быть неявно преобразован в `int` (`true` → 1, `false` → 0) и обратно (любое ненулевое → `true`).

1.1.5 void

Не является типом объекта. Используется для обозначения отсутствия возвращаемого значения и для указателей общего назначения (`void*`).

1.2 Указатели

Указатель — переменная, хранящая адрес объекта в памяти.

```
1 int x = 10;
2 int* ptr = &x;    // & --- взятие адреса
3 *ptr = 20;        // * --- разыменование (доступ к значению по адресу)
```

Как это работает на уровне железа? Указатель — это просто целое число (обычно размером с регистр: 4 байта на 32-битных, 8 на 64-битных системах), которое интерпретируется как адрес ячейки памяти. Разыменование — это инструкция процессора для чтения/записи по данному адресу.

Нулевой указатель (`nullptr` в C++11) — указатель, который гарантированно не указывает ни на один объект. Разыменование `nullptr` — UB.

`void*` — указатель на «что-то». Нельзя разыменовывать (компилятор не знает тип), нельзя к нему применять арифметику. Нужен для совместимости с C API (`malloc` возвращает `void*`).

1.2.1 Адресная арифметика

При прибавлении целого числа к указателю сдвиг происходит на `n * sizeof(T)` байт:

```
1 int arr[5] = {10, 20, 30, 40, 50};
2 int* p = arr;    // p указывает на arr[0]
3 *(p + 2) = 100;   // Эквивалентно arr[2] = 100
4 // p + 2 сдвигает адрес на 2 * sizeof(int) = 8 байт
```

Можно вычитать два указателя одного типа — результат имеет тип `ptrdiff_t` (знаковое целое) и показывает количество элементов между ними.

1.3 Массивы

Массив — непрерывная последовательность элементов одного типа.

```
1 int arr[5] = {1, 2, 3, 4, 5};
2 int arr2[] = {1, 2, 3}; // Размер выводится: 3
```

Array-to-pointer decay: в большинстве контекстов имя массива неявно преобразуется в указатель на первый элемент. Исключения: `sizeof(arr)` (даёт размер всего массива), `&arr` (тип `int(*)[5]`, а не `int**`), `decltype(arr)` (даёт `int[5]`).

Почему `arr[i]` эквивалентно `*(arr+i)`? Потому что оператор `[]` определён именно так. Следствие: `i[arr]` тоже валидно (но так не пишут).

Многомерные массивы: `int m[3][4]` — это массив из 3 элементов, каждый из которых является массивом из 4 `int`. В памяти лежит непрерывно (row-major order).

Массив переменной длины (VLA): в C99 разрешены, в C++ запрещены стандартом (хотя некоторые компиляторы поддерживают как расширение). Вместо VLA — `std::vector`.

1.4 Ссылки (кратко)

Ссылка — это алиас (другое имя) для существующего объекта. В отличие от указателя, ссылку нельзя «переназначить» и она не может быть `null`.

```
1 int x = 10;
2 int& ref = x; // ref --- другое имя для x
3 ref = 20;     // x теперь 20
```

Как реализовано? Внутри компилятор обычно реализует ссылку как указатель, но на уровне языка они отличаются: ссылка не может быть неинициализированной, не может быть `null`, не может быть перенаправлена.

2 Билет 2. Компиляция. Этапы. Препроцессор. ODR. Mangling

2.1 Этапы компиляции

Компиляция C++ программы проходит через несколько этапов:

1. Препроцессинг (cpp / g++ -E):

- Обрабатываются директивы: `#include` (подставляет содержимое файла), `#define` (макросы), `#if/#ifdef/#ifndef` (условная компиляция), `#pragma`.
- Удаляются комментарии.
- Результат — **единица трансляции** (translation unit): один большой текстовый файл.

2. Компиляция (g++ -S):

- Лексический анализ (токены), синтаксический анализ (AST), семантический анализ (проверка типов), оптимизации, генерация ассемблерного кода.
- Каждая единица трансляции компилируется **независимо** от других.

3. Ассемблирование (g++ -c):

- Ассемблерный код → объектный файл (`.o` / `.obj`).
- Содержит машинный код + таблицу символов (имена функций/переменных и их адреса, пока неразрешённые).

4. Линковка (linking):

- Линкер (ld) собирает все объектные файлы и библиотеки в исполняемый файл.
- **Разрешение символов:** если в файле `a.o` вызывается функция `foo()`, определённая в `b.o`, линкер подставляет правильный адрес.
- Если символ не найден — ошибка “undefined reference”. Если найден дважды — ошибка “multiple definition”.

2.2 Директивы препроцессора

2.2.1 #include

Буквально копирует содержимое указанного файла в текущую позицию.

- `#include <file>` — ищет в системных путях.
- `#include "file"` — ищет сначала в текущей директории, потом в системных.

2.2.2 #define и макросы

```
1 #define PI 3.14159
2 #define MAX(a, b) ((a) > (b) ? (a) : (b))
```

Проблемы макросов: нет проверки типов, неожиданные побочные эффекты (`MAX(i++, j)` вычислит `i++` дважды), загрязнение глобального пространства имён. В C++ предпочтительны `constexpr`, `inline` функции и `templates`.

2.2.3 Include Guard

Проблема: если `a.h` включается в `b.h` и `c.h`, а `main.cpp` включает оба, то содержимое `a.h` попадёт дважды → ошибка переопределения.

```
1 // a.h
2 #ifndef A_H_INCLUDED
3 #define A_H_INCLUDED
4 struct Foo { int x; };
5 #endif
```

Альтернатива: `#pragma once` — не стандартизована, но поддерживается всеми основными компиляторами и работает быстрее (не нужно парсить файл заново).

2.3 Объявление vs Определение

- **Объявление** (declaration) — сообщает компилятору, что сущность существует. Не выделяет память.

```
1 extern int x;           // объявление переменной
2 void foo(int a);        // объявление функции
3 class MyClass;          // forward declaration класса
```

- **Определение** (definition) — выделяет память / предоставляет реализацию. Является также объявлением.

```
1 int x = 5;              // определение переменной
2 void foo(int a) { ... } // определение функции
3 class MyClass { ... };  // определение класса
```

Зачем разделение? Потому что единицы трансляции компилируются независимо. Когда `a.cpp` вызывает функцию из `b.cpp`, компилятору `a.cpp` достаточно знать *объявление* (сигнатуру), чтобы проверить типы. Конкретный адрес функции подставит линкер.

2.4 One Definition Rule (ODR)

ODR — фундаментальное правило C++:

1. В одной единице трансляции — не более одного определения любой переменной, функции, класса, перечисления, шаблона.
2. Во всей программе — ровно одно определение каждой ODR-используемой не-inline функции и не-inline переменной.
3. Типы (классы, enum), inline-функции и шаблоны могут определяться в нескольких единицах трансляции, но все определения должны быть **идентичными** (token-by-token).

Нарушение ODR — **UB** (не обязательно ошибка линковки!).

2.5 Связывание (Linkage)

- **Внешнее связывание** (external linkage): имя видно линкеру из других единиц трансляции. По умолчанию у функций и глобальных переменных.
- **Внутреннее связывание** (internal linkage): имя видно только в текущей единице трансляции. Достигается:
 - Ключевым словом `static` перед глобальной переменной/функцией.

- Помещением в анонимное пространство имён (`namespace { ... }`).
- `const` и `constexpr` глобальные переменные по умолчанию имеют внутреннее связывание.
- **Без связывания** (no linkage): локальные переменные.

2.6 inline

Что делает `inline`?

- **Не** гарантирует инлайнинг (вставку тела функции в место вызова). Это лишь *подсказка* компилятору. Современные компиляторы сами решают, что инлайнить.
- **Главное назначение:** изменяет ODR — позволяет определять функцию в заголовочном файле, не нарушая ODR. Линкер при наличии нескольких одинаковых определений `inline`-функции оставит одно.
- С C++17: `inline` переменные — то же самое для переменных (можно определять в header).

2.7 Name Mangling (Декорирование имён)

Проблема: C++ поддерживает перегрузку функций (`foo(int)`, `foo(double)`), пространства имён, шаблоны. Но линкер работает с плоскими символами (именами).

Решение: компилятор кодирует в имя символа информацию о типах параметров, пространстве имён и т.д.:

```
1 // Исходный код:
2 namespace math { int add(int a, int b); }
3 // Может стать символом: _ZN4math3addEii (GCC/Clang)
```

Зачем это знать?

- Ошибки линковки показывают mangled-имена. Утилита `c++filt` декодирует их.
- Разные компиляторы используют разные схемы mangling → объектные файлы от разных компиляторов обычно несовместимы.
- В C нет mangling (нет перегрузки), поэтому `extern "C"` отключает mangling для совместимости.

3 Билет 3. Классы. RAII. Модификаторы. Специальные методы

3.1 Структура класса

`class` и `struct` — почти одно и то же. Единственное отличие: у `struct` по умолчанию `public` доступ, у `class` — `private`.

Класс содержит:

- **Поля** (member variables) — данные.
- **Методы** (member functions) — функции, имеющие доступ к `this`.
- **Вложенные типы, перечисления, typedef/using.**

3.2 Конструкторы и деструкторы

3.2.1 Конструкторы

Вызываются при создании объекта. Задача — привести объект в корректное состояние.

```
1 class Vector {
2     int* data_;
3     size_t size_;
4 public:
5     Vector() : data_(nullptr), size_(0) {}           // default
6     Vector(size_t n) : data_(new int[n]), size_(n) {} // параметризованный
7     Vector(std::initializer_list<int> il);          // initializer_list
8 };
```

3.2.2 Списки инициализации (Member Initializer List)

```
1 Vector(size_t n) : data_(new int[n]), size_(n) {
2     // Тело конструктора
3 }
```

Почему использовать?

- `const` поля и ссылки **обязаны** инициализироваться в списке (нельзя присвоить позже).
- Для полей-объектов: без списка вызовется конструктор по умолчанию, а потом оператор присваивания → двойная работа.
- Порядок инициализации определяется **порядком объявления полей в классе**, а не порядком в списке инициализации!

3.2.3 Деструктор

Вызывается при уничтожении объекта. Задача — освободить ресурсы.

```
1 ~Vector() { delete[] data_; }
```

Деструктор вызывается автоматически при выходе из scope (для объектов на стеке), при `delete` (для объектов в куче), или явно (при placement new).

3.3 RAII (Resource Acquisition Is Initialization)

Идиома: захват ресурса привязан к времени жизни объекта.

- Конструктор захватывает ресурс (выделяет память, открывает файл, блокирует мьютекс).
- Деструктор освобождает ресурс.

Зачем? Гарантирует корректное освобождение ресурсов даже при исключениях. Когда исключение раскручивает стек, деструкторы всех локальных объектов вызываются.

Примеры из стандартной библиотеки: `std::unique_ptr`, `std::shared_ptr`, `std::lock_guard`, `std::fstream`.

3.4 Модификаторы доступа

- `public` — доступен всем.
- `protected` — доступен внутри класса и его наследников.
- `private` — доступен только внутри класса.

- **friend** — предоставляет доступ внешней функции/классу к **private/protected** членам.

Зачем инкапсуляция? Позволяет контролировать инварианты класса: внешний код не может привести объект в некорректное состояние.

3.5 Ключевые слова для членов класса

3.5.1 static

- **Статическое поле:** одно на весь класс (не принадлежит конкретному объекту). Хранится в сегменте данных. Определяется вне класса (до C++17) или с **inline** (C++17).
- **Статический метод:** не имеет **this**, работает только со статическими полями.

3.5.2 explicit

Запрещает неявные преобразования через конструктор:

```
1 class Foo {
2 public:
3     explicit Foo(int x);
4 };
5 Foo f = 42;    // Ошибка! (неявное преобразование запрещено)
6 Foo f(42);    // OK
7 Foo f{42};    // OK
```

Зачем? Предотвращает неожиданные неявные преобразования, которые могут скрывать баги.

3.5.3 const для методов

```
1 int getValue() const { return value_; }
```

В **const**-методе **this** имеет тип **const T***: нельзя менять поля объекта (если поле не **mutable**).
mutable: позволяет изменять поле даже в **const**-методе (пример: кэш, мьютекс).

3.6 Перегрузка операторов

Операторы можно определять как методы класса или свободные функции:

```
1 class Vector {
2 public:
3     Vector operator+(const Vector& rhs) const;    // как метод
4     Vector& operator+=(const Vector& rhs);
5     bool operator==(const Vector& rhs) const;
6     int& operator[](size_t i);                    // неконстантная версия
7     const int& operator[](size_t i) const;        // константная версия
8     friend std::ostream& operator<<(std::ostream& os, const Vector& v);
9 };
```

Некоторые операторы можно определять только как методы: **=**, **[]**, **()**, **->**.

3.7 Специальные методы (Rule of 5)

Компилятор может автоматически генерировать 5 специальных методов:

1. Деструктор: **~T()**
2. Конструктор копирования: **T(const T&)**
3. Оператор присваивания копированием: **T& operator=(const T&)**

4. Конструктор перемещения: `T(T&&)`

5. Оператор присваивания перемещением: `T& operator=(T&&)`

Rule of 5: если вы определяете хотя бы один из этих методов (обычно деструктор), скорее всего, вам нужно определить все пять. Причина: если класс вручную управляет ресурсом, автоматически сгенерированные методы (побитовое/поэлементное копирование) будут некорректны.

Rule of 0: если класс не управляет ресурсами напрямую (использует smart pointers, контейнеры), не определяйте ни одного — полагайтесь на автогенерацию.

`= default` и `= delete`:

```
1 T(const T&) = default; // Попросить компилятор сгенерировать
2 T(const T&) = delete;  // Запретить копирование
```

Таблица автогенерации: если пользователь объявил конструктор перемещения, конструктор копирования не генерируется автоматически (и наоборот). Если объявлен деструктор, конструктор перемещения не генерируется.

3.8 Идиома Copy-and-Swap

Безопасная реализация оператора присваивания:

```
1 class Vector {
2     // ...
3     friend void swap(Vector& a, Vector& b) noexcept {
4         using std::swap;
5         swap(a.data_, b.data_);
6         swap(a.size_, b.size_);
7     }
8
9     Vector& operator=(Vector other) { // Принимаем по значению!
10         swap(*this, other);
11         return *this;
12     }
13 };
```

Почему это хорошо? Строгая гарантия исключений: если конструктор копирования (при передаче по значению) бросает исключение, `*this` не изменён. Работает и для `copy`, и для `move assignment`.

4 Билет 4. Работа с памятью. Стек и куча

4.1 Модель памяти программы

Типичная программа на C++ использует несколько областей памяти:

1. **Сегмент кода** (`text`): исполняемые инструкции.
2. **Сегмент данных:** глобальные и статические переменные (инициализированные в `.data`, неинициализированные в `.bss`).
3. **Стек** (`stack`): локальные переменные, параметры функций, адреса возврата.
4. **Куча** (`heap` / `free store`): динамически выделяемая память.

4.2 Стек (Stack)

Как работает? Стек — область памяти, работающая по принципу LIFO. При вызове функции на стеке создаётся **фрейм** (stack frame), содержащий:

- Локальные переменные
- Параметры функции
- Адрес возврата
- Сохранённые регистры

При выходе из функции фрейм «снимается» — указатель стека (SP) сдвигается.

Характеристики: очень быстрое выделение/освобождение ($O(1)$, это просто сдвиг регистра), ограниченный размер (обычно 1–8 МБ), автоматическое управление.

Stack overflow: при слишком глубокой рекурсии или слишком большом массиве на стеке.

4.3 Куча (Heap / Free Store)

Как работает? Аллокатор (например, `ptmalloc`, `jemalloc`, `tcmalloc`) управляет большим блоком памяти, выданным ОС (через `mmap/sbrk`). Поддерживает списки свободных блоков. При запросе ищет подходящий блок.

Характеристики: медленнее стека (поиск блока, возможная фрагментация), размер ограничен доступной памятью, программист управляет временем жизни.

4.4 new / delete

```
1 int* p = new int(42);           // 1) operator new (выделяет память)
2                                 // 2) конструктор int(42)
3 delete p;                       // 1) деструктор ~int()
4                                 // 2) operator delete (освобождает память)
5
6 int* arr = new int[100];        // operator new[] + конструкторы
7 delete[] arr;                  // деструкторы + operator delete[]
```

Важно: `delete` и `delete[]` — разные операторы! Путать их — UB.

4.5 operator new / operator delete

Это функции, которые `new`-выражение вызывает внутри:

```
1 void* operator new(size_t size);           // глобальный
2 void* operator new(size_t size, const std::nothrow_t&); // не бросает
3 void operator delete(void* ptr) noexcept;
4 void operator delete(void* ptr, size_t size) noexcept; // C++14
```

Можно переопределять для класса или глобально (для профилирования, пулов памяти и т.д.).

4.6 malloc / free (из C)

```
1 int* p = (int*)malloc(sizeof(int) * 10);
2 free(p);
```

Отличия от new/delete:

- Не вызывают конструкторы/деструкторы
- Возвращают `void*` (нужен явный каст)

- При ошибке `malloc` возвращает `nullptr` (не бросает исключение)
- Нельзя смешивать: `malloc + delete` или `new + free` — UB

4.7 Placement new

Позволяет создать объект в **уже выделенной** памяти. Не выделяет память!

```
1 #include <new> // placement new
2
3 alignas(MyClass) char buffer[sizeof(MyClass)];
4 MyClass* obj = new (buffer) MyClass(args...); // вызов конструктора
5 // ...
6 obj->~MyClass(); // ОБЯЗАТЕЛЬНО вызвать деструктор вручную!
7 // НЕ вызывать delete obj! (память не была выделена через new)
```

Зачем? Используется в аллоляторах, контейнерах (`std::vector` выделяет сырую память, потом конструирует элементы), `std::optional`, `std::variant`.

4.8 `std::construct_at` / `std::destroy_at` (C++20)

```
1 std::construct_at(ptr, args...); // Эквивалент placement new
2 std::destroy_at(ptr);           // Эквивалент ptr->~T()
```

Преимущество: работают в `constexpr`-контексте (в отличие от `placement new`).

4.9 Проблемы динамической памяти

- Утечка памяти (memory leak): забыли вызвать `delete`.
- Двойное освобождение (double free): UB.
- Use-after-free: обращение к памяти после `delete`. UB.
- Dangling pointer: указатель на освобождённую память.

Решение: RAII + smart pointers (`unique_ptr`, `shared_ptr`).

5 Билет 5. Исключения

5.1 Механизм исключений

```
1 try {
2     if (error) throw std::runtime_error("Something failed");
3     // ...
4 } catch (const std::runtime_error& e) {
5     std::cerr << e.what() << std::endl;
6 } catch (const std::exception& e) {
7     // Ловит любой наследник std::exception
8 } catch (...) {
9     // Ловит вообще всё
10 }
```

Как работает (Zero-cost exception model)?

- При отсутствии исключений — **нулевые накладные расходы** на исполнение (но увеличивается размер бинарника).

- При `throw` запускается **раскрутка стека** (stack unwinding): последовательно вызываются деструкторы всех локальных объектов во всех фреймах до подходящего `catch`.
- Используется таблица (exception table) в каждой функции, описывающая, какие деструкторы вызывать и какие `catch`-блоки проверять.

`throw;` (без аргумента) — перебрасывает текущее исключение. Работает только внутри `catch`.

5.2 Иерархия стандартных исключений

`std::exception` — базовый класс. Основные наследники:

- `std::logic_error`: `invalid_argument`, `out_of_range`, `length_error`
- `std::runtime_error`: `overflow_error`, `underflow_error`, `range_error`
- `std::bad_alloc` (при неудачном `new`)
- `std::bad_cast` (при неудачном `dynamic_cast`)

5.3 noexcept

```
1 void foo() noexcept;           // Гарантирует, что не бросает
2 void bar() noexcept(true);    // То же
3 void baz() noexcept(false);   // Может бросать (по умолчанию)
4 void qux() noexcept(noexcept(foo())); // Условный noexcept
```

Что происходит, если `noexcept`-функция бросит исключение? Вызывается `std::terminate()` → программа завершается. Стек может не раскручиваться.

Зачем `noexcept`?

- Оптимизации: компилятор может генерировать более эффективный код.
- Контейнеры STL (например, `std::vector`) при реаллокации используют `move`-конструктор только если он `noexcept`. Иначе — копирование (для строгой гарантии).
- Документация: показывает пользователю, что функция безопасна.

5.4 Гарантии безопасности исключений

1. **No-throw guarantee**: операция никогда не бросает исключений. Пример: деструкторы, `swap`, операции с `noexcept`.
2. **Strong guarantee (строгая)**: если операция бросила исключение, состояние программы откатывается к тому, что было до вызова (`commit-or-rollback`). Пример: `std::vector::push_back` (если `move` `noexcept` или используется `copy`).
3. **Basic guarantee (базовая)**: при исключении нет утечек ресурсов, объекты остаются в *валидном, но неопределённом* состоянии. Пример: присваивание без `copy-and-swap`.
4. **No guarantee**: никаких гарантий. Так писать не надо.

Исключения в деструкторах: деструкторы неявно `noexcept`. Если деструктор бросает исключение во время раскрутки стека (когда уже летит другое исключение), вызывается `std::terminate`. **Никогда не бросайте из деструкторов.**

5.5 Альтернативы исключениям

Коды возврата, `std::expected` (C++23), `std::optional`, паттерн `Result`.

6 Билет 6. Шаблоны (Templates)

6.1 Зачем нужны шаблоны?

Шаблоны обеспечивают **обобщённое программирование** — написание кода, работающего с любыми типами, без потери производительности (в отличие от полиморфизма через виртуальные функции).

```
1 template<typename T>
2 T max(T a, T b) {
3     return (a > b) ? a : b;
4 }
5 // Использование:
6 max(1, 2);           // T = int (вывод типа)
7 max<double>(1, 2.5); // Явное указание
```

Как работает? Шаблон — это не функция/класс, а *рецепт* для создания функций/классов. При использовании с конкретным типом компилятор **инстанцирует** (создаёт) конкретную версию. Это происходит во время компиляции.

6.2 Шаблоны функций и классов

```
1 // Шаблон класса
2 template<typename T, size_t N>
3 class Array {
4     T data[N];
5 public:
6     T& operator[](size_t i) { return data[i]; }
7     constexpr size_t size() const { return N; }
8 };
9
10 Array<int, 10> arr; // Инстанцирование
```

Non-type template parameters: параметрами шаблона могут быть не только типы, но и значения (целые числа, указатели, ссылки, с C++20 — литеральные типы):

```
1 template<int N> struct Factorial {
2     static constexpr int value = N * Factorial<N-1>::value;
3 };
4 template<> struct Factorial<0> {
5     static constexpr int value = 1;
6 };
```

6.3 Двухфазный поиск имён (Two-Phase Lookup)

При компиляции шаблона имена ищутся в два этапа:

Фаза 1 (при определении шаблона):

- Проверяется синтаксис.
- Ищутся **non-dependent names** (имена, не зависящие от параметров шаблона).

Фаза 2 (при инстанцировании):

- Ищутся **dependent names** (имена, зависящие от параметра шаблона).
- Выполняется ADL для зависимых имён.

Ключевое слово `typename`:


```

1 template<typename T>
2 void foo() {
3     typename T::value_type x; // Нужно typename, чтобы компилятор
4     // знал, что T::value_type --- это тип, а не статическое поле
5 }

```

Ключевое слово `template` в зависимом контексте:

```

1 template<typename T>
2 void foo(T& obj) {
3     obj.template bar<int>(); // Нужно template перед bar
4 }

```

6.4 Специализация

6.4.1 Полная специализация

```

1 template<>
2 class Array<bool, 8> {
3     // Специальная реализация для bool (bit packing)
4     uint8_t data;
5 };

```

6.4.2 Частичная специализация (только для классов)

```

1 template<typename T>
2 class SmartPtr<T[]> { // Специализация для массивов
3     // ...
4 };
5
6 template<typename T>
7 class Container<T*> { // Специализация для указателей
8     // ...
9 };

```

Для функций частичная специализация **невозможна**. Вместо неё используют перегрузку.

6.5 Специализации функций и перегрузки

```

1 template<typename T>
2 void process(T x) { /* общий случай */ }
3
4 template<>
5 void process<int>(int x) { /* полная специализация */ }
6
7 // Лучше: перегрузка вместо специализации
8 void process(int x) { /* перегрузка для int */ }

```

Почему перегрузка предпочтительнее специализации для функций? Потому что перегрузки участвуют в разрешении перегрузки (overload resolution), а специализации — нет: сначала выбирается базовый шаблон, потом проверяются его специализации. Это может приводить к неожиданному поведению.

6.6 Variadic Templates (шаблоны с переменным числом аргументов)

```

1 // Базовый случай
2 void print() {}
3
4 // Рекурсивный случай
5 template<typename T, typename... Args>
6 void print(T first, Args... rest) {
7     std::cout << first << " ";
8     print(rest...); // Раскрытие пакета
9 }
10
11 print(1, 2.5, "hello"); // T=int, Args={double, const char*}

```

`sizeof...(Args)` — количество аргументов в пакете (вычисляется при компиляции).

6.7 Fold Expressions (C++17)

Лаконичная свёртка пакетов параметров без рекурсии:

```

1 template<typename... Args>
2 auto sum(Args... args) {
3     return (args + ...); // Правая свёртка: a1 + (a2 + (a3 + ...))
4 }
5
6 template<typename... Args>
7 auto sum2(Args... args) {
8     return (... + args); // Левая свёртка: ((a1 + a2) + a3) + ...
9 }
10
11 template<typename... Args>
12 auto sum3(Args... args) {
13     return (0 + ... + args); // Бинарная левая свёртка (с init)
14 }
15
16 template<typename... Args>
17 void printAll(Args... args) {
18     (std::cout << ... << args) << std::endl; // Бинарная левая
19 }

```

Четыре формы:

- `(pack op ...)` — unary right fold
- `(... op pack)` — unary left fold
- `(pack op ... op init)` — binary right fold
- `(init op ... op pack)` — binary left fold

7 Билет 7. Трейты и SFINAE

7.1 Type Traits (трейты типов)

Трейты — шаблонные структуры из `<type_traits>`, дающие информацию о типах на этапе компиляции:

```

1 #include <type_traits>
2 static_assert(std::is_integral_v<int>); // true
3 static_assert(std::is_floating_point_v<double>); // true
4 static_assert(std::is_same_v<int, int32_t>); // зависит от платформы
5 static_assert(std::is_pointer_v<int*>); // true

```

Как устроены внутри? Через специализации шаблонов:

```
1 // Упрощённая реализация is_pointer
2 template<typename T>
3 struct is_pointer : std::false_type {};
4
5 template<typename T>
6 struct is_pointer<T*> : std::true_type {};
7
8 // std::true_type = std::integral_constant<bool, true>
9 // std::false_type = std::integral_constant<bool, false>
```

Трейты-трансформации:

```
1 std::remove_const_t<const int>           // int
2 std::remove_reference_t<int&>             // int
3 std::add_pointer_t<int>                   // int*
4 std::decay_t<const int&>                  // int
5 std::conditional_t<true, int, double>    // int
```

7.2 std::declval

Позволяет «иметь» объект типа T в невычисляемом контексте (decltype, sizeof, noexcept), даже если у T нет конструктора по умолчанию:

```
1 template<typename T, typename U>
2 using sum_type = decltype(std::declval<T>() + std::declval<U>());
3 // Определяет тип результата T + U без создания объектов
```

Как работает? std::declval<T>() возвращает T&&. Вызвать его в рантайме — ошибка линковки (нет определения).

7.3 SFINAE (Substitution Failure Is Not An Error)

Принцип: если при подстановке аргументов в шаблон возникает ошибка в *непосредственном контексте* (immediate context) — это не ошибка компиляции, а просто отбрасывание данной перегрузки.

```
1 // Пример: функция только для целых типов
2 template<typename T>
3 typename std::enable_if<std::is_integral_v<T>, T>::type
4 process(T val) {
5     return val * 2;
6 }
7
8 template<typename T>
9 typename std::enable_if<std::is_floating_point_v<T>, T>::type
10 process(T val) {
11     return val * 2.5;
12 }
```

std::enable_if:

```
1 template<bool B, typename T = void>
2 struct enable_if {};
3
4 template<typename T>
5 struct enable_if<true, T> { using type = T; };
6 // Если B = false, type не существует -> SFINAE
```

7.3.1 Техники применения SFINAE

1. Через возвращаемый тип:

```
1 template<typename T>
2 auto foo(T x) -> std::enable_if_t<std::is_integral_v<T>, void> { ... }
```

2. Через дополнительный параметр шаблона:

```
1 template<typename T, std::enable_if_t<std::is_integral_v<T>, int> = 0>
2 void foo(T x) { ... }
```

3. Через void_t (C++17 idiom):

```
1 template<typename, typename = void>
2 struct has_size : std::false_type {};
3
4 template<typename T>
5 struct has_size<T, std::void_t<decltype(std::declval<T>().size())>>
6       : std::true_type {};
```

`void_t` — это просто:

```
1 template<typename...>
2 using void_t = void;
```

Если выражения внутри валидны → получаем `void` → совпадает со значением по умолчанию второго параметра → выбирается частичная специализация (`true_type`). Если невалидны → SFINAE → выбирается базовый шаблон (`false_type`).

7.4 if constexpr (C++17)

Ветвление на этапе компиляции. Невыбранная ветка **не** инстанцируется:

```
1 template<typename T>
2 auto process(T val) {
3     if constexpr (std::is_integral_v<T>) {
4         return val * 2;
5     } else if constexpr (std::is_floating_point_v<T>) {
6         return val * 2.5;
7     } else {
8         static_assert(false, "Unsupported type"); // C++23 workaround needed
9     }
10 }
```

Отличие от обычного if: обычный `if` требует, чтобы обе ветки компилировались. `if constexpr` отбрасывает ненужную ветку.

7.5 Tag Dispatching

Выбор перегрузки на основе типа-метки:

```
1 template<typename It>
2 void advance_impl(It& it, int n, std::random_access_iterator_tag) {
3     it += n; // O(1)
4 }
5
6 template<typename It>
7 void advance_impl(It& it, int n, std::input_iterator_tag) {
8     for (int i = 0; i < n; ++i) ++it; // O(n)
9 }
10
```

```

11 template<typename It>
12 void advance(It& it, int n) {
13     advance_impl(it, n,
14         typename std::iterator_traits<It>::iterator_category{});
15 }

```

Зачем, если есть `if constexpr` и `SFINAE`? Tag dispatching — это более старая, но очень читаемая техника. Активно используется в STL.

7.6 SFINAE-friendliness

Класс/функция является SFINAE-friendly, если ошибки при невалидных типах возникают в непосредственном контексте (и потому отбрасываются SFINAE), а не глубоко внутри (что было бы `hard error`). Пример: `std::result_of` (C++11) не был SFINAE-friendly, `std::invoke_result` (C++17) — стал.

8 Билет 8. Контейнеры и итераторы

8.1 Контейнеры STL

8.1.1 Последовательные контейнеры

- `std::vector<T>`: динамический массив. Элементы хранятся непрерывно. Вставка в конец — амортизированно $O(1)$ (с реаллокацией). Вставка в середину — $O(n)$. Доступ по индексу — $O(1)$.
- `std::deque<T>`: двусторонняя очередь. Вставка в начало и конец — $O(1)$. Доступ по индексу — $O(1)$. Не гарантирует непрерывность памяти (реализуется как массив указателей на блоки).
- `std::list<T>`: двусвязный список. Вставка/удаление в любом месте (если есть итератор) — $O(1)$. Доступ по индексу — $O(n)$. Плохая `cache locality`.
- `std::forward_list<T>`: односвязный список.
- `std::array<T, N>`: обёртка над C-массивом фиксированного размера. Нет overhead.

8.1.2 Ассоциативные контейнеры (на деревьях)

- `std::set<T>`, `std::map<K, V>`: красно-чёрное дерево. Элементы отсортированы. Поиск/вставка/удаление — $O(\log n)$.
- `std::multiset`, `std::multimap`: допускают дубликаты ключей.

8.1.3 Неупорядоченные контейнеры (хеш-таблицы)

- `std::unordered_set<T>`, `std::unordered_map<K, V>`: хеш-таблица. Поиск/вставка/удаление — амортизированно $O(1)$, в худшем случае $O(n)$.

8.1.4 Контейнеры-адаптеры

- `std::stack<T>`: LIFO (по умолчанию обёртка над `deque`).
- `std::queue<T>`: FIFO.
- `std::priority_queue<T>`: куча (max-heap по умолчанию).

8.2 Итераторы

Итератор — обобщённый указатель: объект, позволяющий обходить элементы контейнера.

8.2.1 Категории итераторов (иерархия)

1. **Input Iterator**: чтение, только вперёд, один проход. Пример: `std::istream_iterator`.
2. **Output Iterator**: запись, только вперёд, один проход. Пример: `std::ostream_iterator`, `std::back_insert_iterator`.
3. **Forward Iterator**: чтение/запись, только вперёд, многократный проход. Пример: `std::forward_list::iterator`.
4. **Bidirectional Iterator**: + движение назад (`--`). Пример: `std::list::iterator`, `std::set::iterator`.
5. **Random Access Iterator**: + произвольный доступ (`it += n`, `it[n]`, сравнение `<`). Пример: `std::vector::iterator`, `std::deque::iterator`.
6. **Contiguous Iterator** (C++17/20): + элементы лежат непрерывно в памяти. Пример: `std::vector::iterator`, `std::array::iterator`, обычный указатель.

Чем отличаются? Набором поддерживаемых операций:

- Input: `*it` (чтение), `++it`, `it == it2`
- Forward: + гарантия многопроходности
- Bidirectional: + `--it`
- Random Access: + `it + n`, `it - n`, `it[n]`, `it < it2`
- Contiguous: + `std::to_address(it)` указывает на непрерывный блок

Зачем категории? Алгоритмы STL выбирают оптимальную реализацию в зависимости от категории: `std::distance` — $O(1)$ для Random Access, $O(n)$ для Input. `std::sort` требует Random Access.

8.3 Range-based for

```
1 for (auto& elem : container) { ... }
2 // Эквивалентно:
3 {
4     auto&& __range = container;
5     auto __begin = std::begin(__range); // или __range.begin()
6     auto __end = std::end(__range);     // или __range.end()
7     for (; __begin != __end; ++__begin) {
8         auto& elem = *__begin;
9         ...
10    }
11 }
```

Как сделать свой класс совместимым? Нужно предоставить `begin()` и `end()` (как методы или через ADL), а итератор должен поддерживать `*`, `++`, `!=`.

8.4 std::iterator_traits

```
1 template<typename It>
2 struct iterator_traits {
3     using value_type = typename It::value_type;
4     using difference_type = typename It::difference_type;
5     using pointer = typename It::pointer;
6     using reference = typename It::reference;
7     using iterator_category = typename It::iterator_category;
8 };
9
10 // Специализация для указателей:
11 template<typename T>
12 struct iterator_traits<T*> {
13     using value_type = T;
14     using difference_type = ptrdiff_t;
15     using pointer = T*;
16     using reference = T&;
17     using iterator_category = random_access_iterator_tag;
18 };
```

8.5 Инвалидация итераторов

При модификации контейнера итераторы могут стать невалидными:

- **vector**: при реаллокации — все итераторы. При вставке/удалении в середину — итераторы после точки вставки.
- **deque**: вставка/удаление в начало/конец инвалидирует все итераторы (но не ссылки/указатели, кроме начала/конца). Вставка в середину — все.
- **list**: инвалидируются только итераторы на удалённые элементы.
- **set/map**: аналогично **list**.
- **unordered_***: при рехешировании — все итераторы.

9 Билет 9. Undefined Behavior (UB)

9.1 Что такое UB?

Undefined Behavior — ситуация, при которой стандарт C++ **не определяет**, что должна делать программа. Результат может быть *любым*: программа может работать “правильно”, упасть, отформатировать диск, или (благодаря оптимизациям компилятора) вести себя совершенно неожиданно.

Почему UB существует? Чтобы дать компилятору свободу для оптимизаций. Например, компилятор может предполагать, что UB никогда не происходит, и удалять проверки.

9.2 Примеры UB

- Разыменование `nullptr` или невалидного указателя
- Переполнение знакового целого (`INT_MAX + 1`)
- Выход за границы массива

- Использование неинициализированной переменной
- Обращение к объекту после `delete` (use-after-free)
- Двойной `delete`
- Нарушение strict aliasing rule
- Модификация `const` объекта через отброс `const`
- Гонка данных (data race) в многопоточной программе
- Бесконечный цикл без побочных эффектов (с C++11, кроме `volatile`)
- Сдвиг на отрицательное число бит или на число $\geq$ ширины типа

9.3 Strict Aliasing Rule

Правило: нельзя обращаться к объекту одного типа через указатель/ссылку на несовместимый тип.

```
1 float f = 3.14f;
2 int* p = reinterpret_cast<int*>(&f);
3 int i = *p; // UB! Нарушение strict aliasing
```

Исключения: через `char*`, `unsigned char*`, `std::byte*` можно читать любой объект (но не наоборот!).

Зачем это правило? Позволяет компилятору оптимизировать: если функция принимает `int*` и `float*`, компилятор может предполагать, что они не указывают на одну и ту же память (не нужно перечитывать).

Правильный способ `reinterpret` переинтерпретации битов:

```
1 float f = 3.14f;
2 int i;
3 std::memcpy(&i, &f, sizeof(i)); // OK!
4 // Или в C++20: std::bit_cast<int>(f);
```

9.4 std::unreachable() (C++23)

Подсказка компилятору, что данная точка в коде никогда не будет достигнута:

```
1 int getValue(int x) {
2     switch (x) {
3         case 0: return 10;
4         case 1: return 20;
5         default: std::unreachable(); // UB если сюда дойдём,
6                                     // но компилятор оптимизирует
7     }
8 }
```

До C++23 использовали `__builtin_unreachable()` (GCC/Clang) или `__assume(false)` (MSVC).

9.5 assert

```
1 #include <cassert>
2 assert(ptr != nullptr); // Если false -> abort()
```

Работает только в отладочной сборке (при `NDEBUG` отключается). Не для обработки ошибок в релизе!

9.6 Санитайзеры

Инструменты для обнаружения ошибок во время выполнения (вставляют проверки в код):

- **AddressSanitizer (ASan)**: выход за границы, use-after-free, double free, утечки. Флаг: `-fsanitize=address`.
- **UndefinedBehaviorSanitizer (UBSan)**: переполнение, сдвиги, null deref, и т.д. Флаг: `-fsanitize=undefined`.
- **ThreadSanitizer (TSan)**: data races. Флаг: `-fsanitize=thread`.
- **MemorySanitizer (MSan)**: чтение неинициализированной памяти. Флаг: `-fsanitize=memory`.

Замедляют программу в 2–10 раз. Используются при разработке и тестировании.

9.7 Отладочная версия стандартной библиотеки

Некоторые реализации STL (libstdc++ в GCC, libc++ в Clang) имеют отладочные режимы, которые проверяют валидность итераторов, выход за границы и т.д. Например: `-D_GLIBCXX_DEBUG` для GCC.

10 Билет 10. Вычисления на этапе компиляции

10.1 constexpr

Для переменных: значение *может* быть вычислено при компиляции (и должно быть, если используется в контексте, требующем compile-time значения: размер массива, template argument):

```
1 constexpr int N = 42;
2 int arr[N]; // OK
```

Для функций: функция *может* вычисляться при компиляции, если все аргументы известны:

```
1 constexpr int factorial(int n) {
2     return n <= 1 ? 1 : n * factorial(n - 1);
3 }
4 constexpr int f5 = factorial(5); // Вычислено при компиляции
5 int x;
6 std::cin >> x;
7 int fx = factorial(x); // Вычислено в рантайме
```

Ограничения constexpr функций (ослабляются с каждым стандартом):

- C++11: только один return, без циклов и переменных
- C++14: циклы, переменные, if-else разрешены
- C++20: можно использовать new/delete, try-catch, виртуальные функции (при условии, что вся динамическая память освобождена до конца вычисления)

10.2 consteval (C++20)

Immediate function — функция, которая **обязана** вычисляться при компиляции. Вызов с runtime-аргументами — ошибка компиляции.

```
1 consteval int square(int n) { return n * n; }
2 constexpr int a = square(5); // OK: 25 при компиляции
3 int x = 5;
4 int b = square(x); // ОШИБКА! x не constexpr
```

Зачем? Гарантировать, что вычисление *точно* произойдёт при компиляции (в отличие от constexpr, где компилятор может отложить на рантайм).

10.3 constexpr (C++20)

Гарантирует **статическую инициализацию** переменной (при компиляции), но не делает переменную const:

```
1 constexpr int global = 42; // Инициализация при компиляции
2 // global можно менять в рантайме!
```

Зачем? Решает проблему **Static Initialization Order Fiasco**: порядок инициализации глобальных переменных из разных единиц трансляции не определён. Если одна зависит от другой — UB. constexpr гарантирует, что инициализация произойдёт при компиляции, до любого рантайм-кода.

10.4 std::is_constant_evaluated() (C++20)

Проверяет, выполняется ли код в контексте вычислений при компиляции:

```
1 constexpr int compute(int x) {
2     if (std::is_constant_evaluated()) {
3         // Компайл-тайм путь (может быть медленнее, но constexpr-совместимый)
4         return slow_constexpr_impl(x);
5     } else {
6         // Рантайм путь (можно использовать SIMD, asm и т.д.)
7         return fast_runtime_impl(x);
8     }
9 }
```

C++23: if consteval { ... } else { ... } — более удобный синтаксис.

Важно: if constexpr (std::is_constant_evaluated()) — **антипаттерн!** Всегда вернёт true, потому что if constexpr сам является constant-evaluated контекстом. Нужно использовать обычный if.

11 Билет 11. Перечислимый тип и тип-объединение

11.1 enum и enum class

11.1.1 enum (C-style)

```
1 enum Color { Red, Green, Blue };
2 // Red, Green, Blue "вытекают" в окружающее пространство имён
3 int x = Red; // Неявное преобразование в int
```

Проблемы: загрязнение пространства имён, неявное преобразование в int (ошибки).

11.1.2 enum class (C++11, scoped enumeration)

```
1 enum class Color : uint8_t { Red, Green, Blue };
2 Color c = Color::Red; // OK
3 // int x = Color::Red; // Ошибка! Нет неявного преобразования
4 int x = static_cast<int>(Color::Red); // Явное преобразование
```

Преимущества: строгая типизация, нет загрязнения namespace, можно указать underlying type.

11.2 union

Все члены делят одну и ту же область памяти. Размер union = размер наибольшего члена.

```

1 union Value {
2     int i;
3     float f;
4     char c;
5 };
6 Value v;
7 v.i = 42;
8 // v.f; // UB! Активный член --- i, а не f (кроме char)

```

Проблемы: нет информации о том, какой член сейчас активен. Нельзя хранить типы с нетривиальными конструкторами/деструкторами (до C++11).

С C++11 можно, но нужно вручную вызывать placement new и деструкторы:

```

1 union S {
2     std::string str;
3     int i;
4     S() : i(0) {}
5     ~S() {} // Нужно знать, какой член активен!
6 };

```

union-like class: класс, у которого хотя бы один анонимный union как поле.

11.3 std::variant (C++17)

Типобезопасный union. Хранит одно из перечисленных значений и знает, какое именно.

```

1 std::variant<int, double, std::string> v;
2 v = 42; // Хранит int
3 v = 3.14; // Теперь double
4 v = std::string("hello"); // Теперь string
5
6 // Доступ:
7 std::get<int>(v); // Бросает std::bad_variant_access
8 std::get<2>(v); // По индексу. "hello"
9 auto* p = std::get_if<int>(&v); // Возвращает nullptr или указатель
10
11 v.index(); // Индекс текущего типа (0, 1 или 2)
12 std::holds_alternative<int>(v); // false

```

Размер: $\text{sizeof}(\text{variant}) = \max(\text{sizeof}(\text{типов})) + \text{overhead}$ для хранения индекса (обычно 1-4 байта) + выравнивание.

11.4 std::visit

Паттерн Visitor для variant:

```

1 std::variant<int, double, std::string> v = 42;
2
3 std::visit([](auto&& arg) {
4     using T = std::decay_t<decltype(arg)>;
5     if constexpr (std::is_same_v<T, int>) {
6         std::cout << "int: " << arg;
7     } else if constexpr (std::is_same_v<T, double>) {
8         std::cout << "double: " << arg;
9     } else {
10        std::cout << "string: " << arg;
11    }
12 }, v);

```

Как работает? Внутри генерируется таблица (массив указателей на функции или switch) для вызова правильной перегрузки в зависимости от текущего индекса.

Overloaded pattern (удобный visit):

```
1 template<typename... Ts> struct overloaded : Ts... {
2     using Ts::operator()...;
3 };
4 template<typename... Ts> overloaded(Ts...) -> overloaded<Ts...>;
5
6 std::visit(overloaded{
7     [](int i) { std::cout << "int: " << i; },
8     [](double d) { std::cout << "double: " << d; },
9     [](const std::string& s) { std::cout << "string: " << s; }
10 }, v);
```

11.5 std::monostate

Тип без данных, defaultconstructible. Используется как первый тип в `variant`, чтобы он мог быть default-constructible, даже если ни один из “настоящих” типов таковым не является:

```
1 struct NoDefault { NoDefault(int) {} };
2 // std::variant<NoDefault> v; // Ошибка! NoDefault не default-constructible
3 std::variant<std::monostate, NoDefault> v; // OK, по умолчанию monostate
```

11.6 valueless_by_exception

Если во время присваивания нового значения в `variant` конструктор бросил исключение, `variant` может оказаться в состоянии “без значения”:

```
1 v.valueless_by_exception(); // true, если variant пуст
2 v.index(); // variant::npos
```

12 Билет 12. Rvalue ссылки и семантика перемещения

12.1 Категории значений

Каждое выражение в C++ имеет две характеристики: тип и категорию значения.

- **lvalue** (locator value): выражение, обозначающее объект с устойчивым местом в памяти. Можно взять адрес. Примеры: переменные (`x`), `*ptr`, `arr[i]`, вызов функции, возвращающей `T&`.
- **prvalue** (pure rvalue): “чистое” временное значение, не имеет адреса. Примеры: литералы (`42`), `x + y`, вызов функции, возвращающей `T` (не ссылку).
- **xvalue** (eXpiring value): объект, ресурсы которого можно “украсть”. Примеры: `std::move(x)`, вызов функции, возвращающей `T&&`.

glvalue = **lvalue** $\cup$ **xvalue** (имеет identity).

rvalue = **prvalue** $\cup$ **xvalue** (можно перемещать).

12.2 Rvalue ссылки

```
1 int&& r = 42;           // OK: rvalue ссылка на временный объект
2 int x = 10;
3 // int&& r2 = x;        // Ошибка! x --- lvalue
4 int&& r3 = std::move(x); // OK: std::move превращает в xvalue
```

Важно: сама переменная `r` (и `r3`) — это **lvalue**! Она имеет имя и адрес. Это распространённая ловушка.

12.3 Семантика перемещения

Зачем? При копировании тяжёлых объектов (вектор из миллиона элементов) выполняется дорогое копирование данных. Если исходный объект больше не нужен (временный или после `std::move`), можно “украсть” его внутренние данные вместо копирования.

```
1 class Vector {
2     int* data_;
3     size_t size_;
4 public:
5     // Конструктор перемещения
6     Vector(Vector&& other) noexcept
7         : data_(other.data_), size_(other.size_) {
8         other.data_ = nullptr; // "Обнуляем" источник
9         other.size_ = 0;
10    }
11
12    // Оператор присваивания перемещением
13    Vector& operator=(Vector&& other) noexcept {
14        if (this != &other) {
15            delete[] data_;
16            data_ = other.data_;
17            size_ = other.size_;
18            other.data_ = nullptr;
19            other.size_ = 0;
20        }
21        return *this;
22    }
23 };
```

Состояние после перемещения: объект должен быть в *валидном, но неопределённом* состоянии. Обычно — “пустой”. Деструктор должен корректно работать.

12.4 std::move

`std::move(x)` — это просто `static_cast<T&&>(x)`. **Не перемещает ничего!** Просто превращает **lvalue** в **xvalue**, что позволяет выбрать конструктор перемещения при перегрузке.

```
1 std::vector<int> a = {1, 2, 3};
2 std::vector<int> b = std::move(a);
3 // Теперь b содержит {1, 2, 3}, a --- пуст (но валиден)
```

12.5 Избегание копирования: RVO и NRVO

RVO (Return Value Optimization) — компилятор конструирует возвращаемый объект непосредственно в месте вызова, минуя копирование/перемещение:

```

1 std::vector<int> createVector() {
2     return std::vector<int>{1, 2, 3}; // RVO: нет копирования
3 }

```

NRVO (Named RVO) — то же для именованных переменных:

```

1 std::vector<int> createVector() {
2     std::vector<int> v{1, 2, 3};
3     return v; // NRVO: компилятор может опустить перемещение
4 }

```

С C++17 RVO для prvalue **обязательна** (mandatory copy elision): `return T(args)` гарантированно не копирует. NRVO остаётся необязательной оптимизацией.

Не пишите `return std::move(v);`! Это запрещает NRVO.

12.6 Классы-агрегаты

Агрегат (C++17) — это класс без:

- Пользовательских конструкторов
- Private/protected нестатических полей
- Виртуальных функций
- Виртуальных/private/protected базовых классов

Агрегаты можно инициализировать фигурными скобками:

```

1 struct Point { int x, y; };
2 Point p{1, 2}; // Aggregate initialization

```

12.7 Ref-спецификация методов

Можно перегружать методы для lvalue и rvalue объектов:

```

1 class Foo {
2     std::string data_;
3 public:
4     const std::string& getData() const& { return data_; } // lvalue
5     std::string getData() && { return std::move(data_); } // rvalue
6 };
7
8 Foo f;
9 auto s1 = f.getData(); // копирование
10 auto s2 = Foo().getData(); // перемещение

```

13 Билет 13. Perfect Forwarding

13.1 Проблема

Хотим написать обёртку, которая передаёт аргументы другой функции, **сохраняя** их категорию значения (lvalue/rvalue) и cv-квалификацию:

```

1 template<typename T>
2 void wrapper(T arg) { // По значению --- всегда копирует
3     foo(arg);
4 }
5 template<typename T>

```

```

6 void wrapper(T& arg) {    // По lvalue-ссылке --- не работает с rvalue
7     foo(arg);
8 }

```

13.2 Universal Reference (Forwarding Reference)

```

1 template<typename T>
2 void wrapper(T&& arg) {    // Это НЕ rvalue ссылка!
3     // Это "universal reference" (forwarding reference)
4 }

```

Forwarding reference возникает ТОЛЬКО когда:

1. Тип объявлен как T&&, где T — **выводимый** шаблонный параметр.
2. Или в auto&& x =

Не являются forwarding references: const T&&, vector<T>&&, конкретные типы int&&.

13.3 Reference Collapsing (сворачивание ссылок)

В C++ нельзя иметь “ссылку на ссылку”, но при выводе типов шаблонов они могут возникать. Тогда применяется сворачивание:

T	T&&	Результат
int&	int& &&	int&
int&&	int&& &&	int&&
int	int&&	int&&

Правило: если хотя бы одна ссылка — lvalue (&), результат — &. Иначе — &&.

13.4 Правило вывода типов для forwarding reference

```

1 template<typename T>
2 void foo(T&& arg);
3
4 int x = 42;
5 foo(x);    // x --- lvalue => T = int&,   arg : int& && = int&
6 foo(42);   // 42 --- rvalue => T = int,   arg : int&&

```

13.5 std::forward

```

1 template<typename T>
2 void wrapper(T&& arg) {
3     foo(std::forward<T>(arg));
4     // Если T = int&   => forward<int&>(arg) => lvalue (int&)
5     // Если T = int     => forward<int>(arg)  => rvalue (int&&)
6 }

```

Как реализован std::forward?

```

1 template<typename T>
2 T&& forward(std::remove_reference_t<T>& arg) noexcept {
3     return static_cast<T&&>(arg);
4 }

```

Если T = int&: int& && = int& — возвращает lvalue.

Если T = int: int&& — возвращает rvalue.

13.6 Perfect Backwarding

Проблема “обратной пересылки” — сохранить категорию значения при *возврате* из функции:

```
1 template<typename F, typename... Args>
2 decltype(auto) wrapper(F&& f, Args&&... args) {
3     return std::forward<F>(f)(std::forward<Args>(args)...);
4 }
```

`decltype(auto)` сохраняет ссылочность: если `f` возвращает `int&`, `wrapper` тоже вернёт `int&`.

13.7 decltype и trailing return type

`decltype(expr)`: возвращает тип выражения.

- Если `expr` — имя переменной (id-expression): возвращает объявленный тип.
- Если `expr` — другое выражение: возвращает тип + ссылочность в зависимости от категории (T для prvalue, T& для lvalue, T&& для xvalue).

```
1 int x = 0;
2 decltype(x)    // int (id-expression)
3 decltype((x))  // int& (lvalue expression в скобках)
```

Trailing return type:

```
1 template<typename T, typename U>
2 auto add(T a, U b) -> decltype(a + b) {
3     return a + b;
4 }
```

13.8 auto и decltype(auto)

- `auto`: выводит тип по правилам вывода шаблонных параметров (отбрасывает ссылки и cv-qualifiers верхнего уровня).
- `decltype(auto)`: выводит тип по правилам `decltype` (сохраняет ссылки и cv-qualifiers).

```
1 int x = 0;
2 int& rx = x;
3 auto a = rx;           // int (ссылка отброшена)
4 decltype(auto) b = rx; // int& (ссылка сохранена)
```

14 Билет 14. Наследование и Полиморфизм

14.1 Наследование

```
1 class Base {
2 public:
3     int x;
4     void foo();
5 };
6
7 class Derived : public Base {
8 public:
9     int y;
10    void bar();
11 };
```


Виды наследования:

- public: public $\rightarrow$ public, protected $\rightarrow$ protected.
- protected: public, protected $\rightarrow$ protected.
- private: public, protected $\rightarrow$ private.

Для struct по умолчанию public, для class — private.

14.2 Slicing (срезка)

```
1 Derived d;  
2 Base b = d; // Срезка! Часть Derived "обрезается"  
3 // b содержит только поля Base
```

Как избежать? Работать через указатели или ссылки на базовый класс:

```
1 Derived d;  
2 Base& ref = d; // OK, нет срезки  
3 Base* ptr = &d; // OK
```

14.3 Статический и динамический тип

```
1 Base* ptr = new Derived();  
2 // Статический тип ptr: Base*  
3 // Динамический тип ptr: Derived*
```

Статический тип определяет, какие методы доступны при компиляции. Динамический тип определяет, какой виртуальный метод будет вызван в рантайме.

14.4 Виртуальные функции

```
1 class Base {  
2 public:  
3     virtual void speak() { std::cout << "Base\n"; }  
4 };  
5  
6 class Derived : public Base {  
7 public:  
8     void speak() override { std::cout << "Derived\n"; }  
9 };  
10  
11 Base* ptr = new Derived();  
12 ptr->speak(); // "Derived" --- динамический полиморфизм!
```

Как работает? Через VTable (Virtual Table / таблицу виртуальных методов):

- Каждый класс с виртуальными функциями имеет **VTable** — массив указателей на функции.
- Каждый объект такого класса содержит скрытый указатель **vptr** на VTable своего класса.
- При вызове виртуальной функции: `ptr->vptr->speak()` (два уровня косвенности).
- **override** — проверяет, что функция действительно переопределяет виртуальную из базы.
final — запрещает дальнейшее переопределение.

Стоимость: один дополнительный указатель в каждом объекте (`sizeof` увеличивается), косвенный вызов (не инлайнится обычно), но overhead мал.

14.5 Чисто виртуальные функции и абстрактные классы

```
1 class Shape {
2 public:
3     virtual double area() const = 0; // Чисто виртуальная
4     virtual ~Shape() = default;
5 };
6 // Shape --- абстрактный класс, нельзя создать экземпляр
```

Класс с хотя бы одной чисто виртуальной функцией — **абстрактный**. Наследник должен реализовать все чисто виртуальные функции, иначе тоже абстрактный.

14.6 Виртуальный деструктор

```
1 Base* ptr = new Derived();
2 delete ptr; // Если деструктор Base НЕ виртуальный --- UB!
```

Правило: если класс имеет хотя бы одну виртуальную функцию (т.е. предназначен для полиморфного использования), деструктор **должен** быть виртуальным.

Если класс не предназначен для наследования — можно не делать деструктор виртуальным (экономит vptr).

14.7 dynamic_cast и RTTI

```
1 Base* ptr = getShape();
2 if (Derived* d = dynamic_cast<Derived*>(ptr)) {
3     // ptr указывает на Derived (или его наследника)
4     d->derivedMethod();
5 } else {
6     // ptr не указывает на Derived
7 }
8
9 Base& ref = getShapeRef();
10 Derived& d = dynamic_cast<Derived&>(ref); // Бросает std::bad_cast при неудаче
```

Как работает? Использует **RTTI** (Run-Time Type Information) — информацию о типе, хранящуюся вместе с vtable. Поэтому требует виртуальных функций.

typeid:

```
1 #include <typeinfo>
2 Base* ptr = new Derived();
3 std::cout << typeid(*ptr).name(); // Имя Derived (mangled)
```

14.8 Empty Base Optimization (EBO)

Пустой класс имеет `sizeof ≥ 1` (чтобы разные объекты имели разные адреса). Но при наследовании от пустого класса, его размер может быть оптимизирован до 0:

```
1 struct Empty {};
2 struct Derived : Empty {
3     int x;
4 };
5 static_assert(sizeof(Derived) == sizeof(int)); // EBO!
```

Зачем? Используется в STL: аллокатор, компаратор часто — пустые классы. Без EBO каждый контейнер был бы на 1+ байт больше.

14.9 [[no_unique_address]] (C++20)

Альтернатива EBO без наследования:

```
1 struct Container {
2     [[no_unique_address]] Allocator alloc; // Может занимать 0 байт
3     int* data;
4 };
```

14.10 Множественное наследование

```
1 class A { public: int a; };
2 class B { public: int b; };
3 class C : public A, public B {
4     // Содержит и a, и b
5 };
```

Проблема алмаза (diamond problem):

```
1 class Base { public: int x; };
2 class A : public Base {};
3 class B : public Base {};
4 class C : public A, public B {};
5 // C содержит ДВЕ копии Base::x!
6 // c.x --- неоднозначно. Надо c.A::x или c.B::x
```

14.11 Виртуальное наследование

Решение проблемы алмаза:

```
1 class A : virtual public Base {};
2 class B : virtual public Base {};
3 class C : public A, public B {};
4 // C содержит ОДНУ копию Base
```

Как работает? При виртуальном наследовании подобъект `Base` не вкладывается напрямую в `A` и `B`. Вместо этого хранится указатель/смещение к общему подобъекту `Base`. Это добавляет overhead.

Конструктор `Base` при виртуальном наследовании вызывается **самым производным классом** (`C`), а не промежуточными (`A`, `B`).

15 Билет 15. Пространства имён

15.1 namespace

```
1 namespace math {
2     int add(int a, int b) { return a + b; }
3     namespace detail { // Вложенный
4         void helper() {}
5     }
6 }
7 math::add(1, 2);
8 math::detail::helper();
9
10 // C++17: вложенные namespace
11 namespace math::detail { ... }
```

15.2 Namespace alias

```
1 namespace fs = std::filesystem;
2 fs::path p("/home");
```

15.3 using-объявление vs using-директива

```
1 using std::cout;           // using-объявление: вносит ОДНО имя
2 cout << "hello";
3
4 using namespace std;       // using-директива: вносит ВСЕ имена из std
5 // Не рекомендуется в заголовочных файлах!
```

Почему `using namespace` опасен в `header`? Все файлы, включающие этот `header`, получают загрязнённое глобальное пространство имён, что может привести к конфликтам.

15.4 ADL (Argument Dependent Lookup / Koenig Lookup)

При вызове неквалифицированной функции компилятор **дополнительно** ищет её в пространствах имён, ассоциированных с типами аргументов:

```
1 namespace N {
2     struct Foo {};
3     void bar(Foo) {}
4 }
5
6 N::Foo f;
7 bar(f); // Находит N::bar через ADL! (не нужно N::bar(f))
```

Зачем ADL?

- Операторы: `std::cout << "hello"` — `operator<<` находится в `std`, но ADL его находит через тип `std::ostream`.
- `swap`: идиома `using std::swap; swap(a, b);` — сначала ADL ищет пользовательский `swap`, потом `std::swap` как fallback.
- `begin/end`: аналогично.

15.5 Безымянные (анонимные) пространства имён

```
1 namespace {
2     int internal_var = 42;
3     void internal_func() {}
4 }
5 // internal_var и internal_func видны только в этом файле
6 // Эквивалент static на уровне файла, но работает и для типов
```

Предпочтительнее `static` для ограничения видимости, потому что:

- Работает для классов, enum, typedef (`static` — только для функций и переменных).
- Более идиоматично в C++.

16 Билет 16. Статический и динамический полиморфизм. Лямбды

16.1 Статический полиморфизм

Реализуется через шаблоны. Разрешается на этапе компиляции. Нет overhead в рантайме (нет vtable, вызовы инлайнятся).

```
1 template<typename Shape>
2 double getArea(const Shape& s) {
3     return s.area(); // Компилятор подставит конкретный тип
4 }
```

CRTP (Curiously Recurring Template Pattern):

```
1 template<typename Derived>
2 class Base {
3 public:
4     void interface() {
5         static_cast<Derived*>(this)->implementation();
6     }
7 };
8
9 class Concrete : public Base<Concrete> {
10 public:
11     void implementation() { ... }
12 };
```

16.2 Динамический полиморфизм

Через виртуальные функции. Разрешается в рантайме. Позволяет хранить объекты разных типов в одном контейнере (через указатели на базу).

16.3 Сравнение

	Статический	Динамический
Когда разрешается	Компиляция	Рантайм
Overhead	Нет	vtable, vptr
Инлайнинг	Да	Обычно нет
Гетерогенный контейнер	Нет	Да
Время компиляции	Дольше	Короче
Размер бинарника	Больше (инстанцирование)	Меньше

16.4 Лямбда-выражения

```
1 auto lambda = [capture](params) -> ret_type { body };
```

16.4.1 Списки захвата

```
1 int x = 10, y = 20;
2 auto l1 = [x]() { return x; }; // Копирование x
3 auto l2 = [&x]() { x++; }; // Ссылка на x
4 auto l3 = [=]() { return x + y; }; // Все по копии
5 auto l4 = [&]() { x++; y++; }; // Все по ссылке
6 auto l5 = [=, &x]() { x++; return y; }; // y по копии, x по ссылке
```

```

7 auto l6 = [x = std::move(vec)]() {}; // Init capture (C++14)
8 auto l7 = [*this]() {}; // Копия *this (C++17)

```

16.4.2 Как лямбда устроена внутри?

Компилятор генерирует **анонимный класс** (closure type):

```

1 // auto lambda = [x](int y) { return x + y; };
2 // Эквивалентно:
3 class __lambda_42 {
4     int x;
5 public:
6     __lambda_42(int x) : x(x) {}
7     int operator()(int y) const { return x + y; }
8 };
9 auto lambda = __lambda_42(x);

```

16.4.3 Свойства типа лямбды

- Каждая лямбда имеет **уникальный тип** (даже две одинаковые лямбды — разные типы).
- `operator()` по умолчанию `const`. Добавьте `mutable`, чтобы изменять захваченные по копии переменные.
- Лямбда без захвата может неявно преобразоваться в указатель на функцию:

```

1 void (*fp)(int) = [](int x) { std::cout << x; }; // OK

```

- С C++20: лямбды могут быть `default-constructible` и `assignable` (если без захвата).

16.4.4 Generic лямбды (C++14)

```

1 auto add = [](auto a, auto b) { return a + b; };
2 // Эквивалентно шаблонному operator()

```

16.4.5 Шаблонные лямбды (C++20)

```

1 auto lambda = []<typename T>(std::vector<T>& v) {
2     // Знаем тип T
3 };

```

16.4.6 Лямбды в невычисляемых контекстах (C++20)

```

1 using F = decltype([](int x) { return x; }); // OK с C++20

```

17 Билет 17. Библиотеки и ABI

17.1 Виды библиотек

17.1.1 Header-only

Вся реализация в заголовочных файлах. Пользователь просто делает `#include`.

Плюсы: простота использования, нет проблем с линковкой, полная инлайнизация.

Минусы: увеличивает время компиляции каждого файла, который включает header.

Примеры: Eigen, nlohmann/json, Catch2.

17.1.2 Статические библиотеки (.a / .lib)

Архив объектных файлов. При линковке нужные объектные файлы **копируются** в исполняемый файл.

Плюсы: нет зависимости от внешних файлов в рантайме, возможна оптимизация ЛТО.

Минусы: увеличивает размер исполняемого файла, обновление библиотеки требует перекомпиляции.

17.1.3 Динамические библиотеки (.so / .dll / .dylib)

Загружаются в рантайме.

Плюсы: экономия памяти (одна копия для всех процессов), обновление без перекомпиляции.

Минусы: зависимость от наличия .so/.dll, небольшой overhead при вызове (PLT/GOT), проблемы совместимости ABI.

```
1 // Windows: экспорт/импорт символов
2 #ifdef MYLIB_EXPORTS
3     #define MYLIB_API __declspec(dllexport)
4 #else
5     #define MYLIB_API __declspec(dllimport)
6 #endif
7
8 class MYLIB_API MyClass { ... };
```

На Linux по умолчанию все символы экспортируются. Можно ограничить через `-fvisibility=hidden` и `__attribute__((visibility("default")))`.

17.2 ABI (Application Binary Interface)

ABI определяет:

- Размеры и выравнивание типов (data layout)
- Порядок полей в структуре (padding)
- Calling convention (как передаются аргументы: через регистры или стек, кто очищает стек)
- Name mangling scheme
- Формат vtable
- Формат исключений

Проблема ABI-совместимости: если библиотека скомпилирована одним компилятором (или его версией), а клиентский код — другим, и их ABI несовместимы, будут неопределённые ошибки в рантайме.

Примеры несовместимости:

- GCC и MSVC имеют разные ABI (разный mangling, vtable layout).
- GCC 4.x и GCC 5.x поменяли ABI для `std::string` (SSO).

17.3 extern “C”

```
1 extern "C" {
2     int add(int a, int b); // Компилируется без name mangling
3 }
```

Зачем?

- Для совместимости с C-библиотеками (C не знает про mangling).
- Для создания библиотек, которые можно вызывать из других языков (Python, Rust и т.д.).

Ограничения: нет перегрузки (одно имя = один символ), нет классов, шаблонов и т.д. через extern "C" интерфейс.

17.4 PImpl (Pointer to Implementation)

Идиома сокрытия деталей реализации. Также называется “d-pointer” (Qt), “compiler firewall”, “opaque pointer”.

```

1 // widget.h
2 class Widget {
3 public:
4     Widget();
5     ~Widget();
6     void doSomething();
7 private:
8     struct Impl;           // Forward declaration
9     std::unique_ptr<Impl> pImpl_; // Указатель на реализацию
10 };
11
12 // widget.cpp
13 struct Widget::Impl {
14     std::string name;
15     std::vector<int> data;
16     // ... сложные зависимости
17 };
18
19 Widget::Widget() : pImpl_(std::make_unique<Impl>()) {}
20 Widget::~~Widget() = default; // Определён в .cpp, где Impl полный тип!
21 void Widget::doSomething() { pImpl_->data.push_back(42); }
```

Зачем?

1. **Ускорение компиляции:** изменение деталей реализации (Impl) не требует перекомпиляции клиентского кода (header не изменился).
2. **Бинарная совместимость:** размер Widget (один указатель) не меняется при изменении Impl → можно обновлять динамическую библиотеку без перекомпиляции клиента.
3. **Соккрытие зависимостей:** <string>, <vector> нужны только в .cpp, а не в header.

Минусы: дополнительная аллокация в куче, косвенный доступ к данным, не инлайнятся методы.

17.5 Наследование при PImpl

Если нужен полиморфизм:

```

1 // base.h
2 class Base {
3 public:
4     virtual ~Base();
5     void doSomething();
6 protected:
7     struct Impl;
8     std::unique_ptr<Impl> pImpl_;
```



```
9     Base(std::unique_ptr<Impl> impl);
10 };
11
12 // derived.h
13 class Derived : public Base {
14 public:
15     Derived();
16     ~Derived() override;
17 private:
18     struct DerivedImpl;
19 };
```